

MicroAPI Guard: Real-Time API Anomaly Detection Using ML and DL

A synopsis submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Information Technology

by

Sunny Surendra Niraml C32

Tausib Samir Patel C37

Yash Nandkumar Patil C54

Guided by

Mrs. Priyanka Kumbhar

Asst. Prof., Dept. of IT



G H Raison College of Engineering and Management, Pune

Academic Year 2026–27

Winter 2026

Declaration

We hereby declare that the project synopsis entitled “**MicroAPI Guard: Real-Time API Anomaly Detection Using ML and DL**” submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology in Information Technology is our own work and has not been submitted previously for any degree.

We further declare that all sources of information have been acknowledged by proper references.

Submitted by:

Sunny Surendra Nirmal
B.Tech IT C32

Tausib Samir Patel
B.Tech IT C37

Yash Nandkumar Patil
B.Tech IT C54

Date: _____

Contents

1	Abstract	1
2	Introduction	3
3	Objectives	5
4	Literature Review	6
5	System Architecture	13
6	Problem Statement	16
7	Proposed Methodology	17
7.1	Data Collection, Preprocessing, and Model Training	17
7.1.1	Data Collection	18
7.1.2	Preprocessing	18
7.1.3	Calibration Phase	19
7.1.4	Real-Time Inference	19
7.2	Technology Stack	20
8	Project Plan and Timeline	21
9	Expected Outcome	24
10	Conclusion	26
	References	27

1 Abstract

This project proposes design and develop a deployable API security gateway that would perform a learned stacking ensemble operation on an array of models trained to distinguish between legitimate and attacking traffic. Unlike traditional API gateways that perform some rudimentary detection based on rate-limiting or signature matching, the developed solution performed three stages of detection and isolation in parallel, one statistical rate limiter, one machine learning-based anomaly detection model, and one deep learning-based anomaly detection model. The latter two, Isolation Forest and Autoencoder, provided a score for the degree of abnormality of a given request, which along with the rate-limiting information, was processed by logistic regression as the final stage in the stacking ensemble to indicate whether the request was normal or anomalous. Critically, all models were trained on the validation set; hence, each request had passed through all of the detectors to determine the final label. The models were trained on the traffic sent to the FastAPI server, which was attacked using the Locust load-testing framework.

After training, the FastAPI server gateway loaded the models' weights in memory and could process downstream traffic in near real-time, assigning scores and labels with minimal added latency per request. This was achieved using Redis to perform the sliding-window computation for the rate limiter. The project also featured a Streamlit dashboard that visualized the output of the detectors in real-time, highlighting which one of them had raised an alert and therefore contributing to the final label. As such, it became possible to identify which particular feature or features had triggered the model. In practice, upon deployment on a new server, the gateway would first calibrate itself, learning the characteristics of legitimate traffic before switching to the detection mode.

We designed this project as a proof of concept, demonstrating that an API security gateway could be built as a deployable micro-service that would provide real-time protection by using a machine learning-based anomaly detection model. By prioritizing simplicity, the project achieved decent performance and accuracy, being sufficient for the task at hand, while also being straightforward to deploy in production. As such, this project became an exercise in applied machine learning, where We could demonstrate the ability to train a model to detect anomalies in the API traffic and evaluate its performance. It also provided an opportunity to show how such a model could be applied in practice, protecting production systems. In this project, We also demonstrated the versatility of the same model by applying it to different backend systems with minimal calibration, highlighting the relative ease of deployment in a different production environment. Specifically, recalibrating the model was sufficient in most cases, as opposed to retraining it from scratch.

KeyWords- API Security, Anomaly Detection, Machine Learning, Deep Learning, Stacking Ensemble, Isolation Forest, Autoencoder, Rate Limiting, Model Calibration.

2 Introduction

In recent years, the shift toward microservice architectures has fundamentally changed how applications are built. Instead of monolithic programs, modern applications are now sets of small, independent services communicating via REST APIs. While this brings flexibility and scalability, it also expands the attack surface. Every API endpoint is a potential entry point, and with hundreds of services making thousands of calls per second, manual review is impossible. Intelligent API inspection mechanisms are no longer optional; they are a necessity.

Most traditional API protection tools rely on static rate-limiting or firewall-like rules. Static quotas—like blocking an IP after 100 requests per minute—are easily bypassed by attackers distributing requests across multiple clients. Firewalls can block large-scale DDoS attacks, but they require constant manual updates as new threats emerge. Overall, these traditional mechanisms are too rigid to handle the dynamic nature of large-scale microservice applications.

As API infrastructures grow, enterprises often turn to commercial solutions like Wallarm or AWS WAF. While these tools offer better protection, they come with a steep price curve and act as closed-source black boxes. Their detection methodologies are not transparent, making it hard for researchers to validate or improve them. Consequently, there is a strong need for an open, adaptive mechanism that can learn legitimate traffic patterns and detect outliers without relying on proprietary code.

Machine learning has made significant leaps in anomaly detection. Instead of relying on hardcoded signatures, unsupervised learning algorithms can cluster data points and identify outliers that indicate malicious activity. Techniques like the Isolation Forest identify anomalies using tree-based isolation, while Autoencoders detect them by measuring reconstruction errors. These approaches have proven highly effective in network security, but they are rarely evaluated together under a unified, open framework.

Modern development practices now allow these models to run efficiently. The Python FastAPI framework can handle thousands of asynchronous requests per second with minimal overhead. When paired with an in-memory database like Redis, it becomes possible to implement sliding-window algorithms that track real-time client behavior. Together, these technologies make real-time, ML-driven API inspection computationally viable.

This project develops MicroAPI Guard, an open-source API protection framework designed to fill this gap. The system intercepts incoming HTTP requests and parses them through a feature extraction module, analyzing both payload characteristics and

real-time client behavior. These features are sent to three parallel base detectors: Static Rules, an Isolation Forest, and a Deep Autoencoder. Each generates an anomaly score. A Logistic Regression meta-learner then dynamically combines these scores to produce a final Normal/Anomaly prediction.

Crucially, unlike passive monitoring tools, this framework performs inline protection. If a request is flagged as an anomaly, the gateway immediately blocks it with an HTTP 403 Forbidden response before it reaches the backend. Otherwise, legitimate traffic is forwarded without interruption.

To ensure it works across different environments, the prototype includes a lightweight calibration phase. By observing just a few minutes of normal traffic on a new backend, the system automatically adjusts its statistical thresholds and adapts the pre-trained models. This allows the framework to be plugged in front of any REST-based architecture—whether it’s built in FastAPI, Java Spring Boot, or Node.js—without modifying the backend source code.

Finally, the framework includes a Streamlit dashboard that visualizes analytics in real-time. It displays live traffic statistics, individual model scores for explainability, and whether requests were forwarded or blocked. Altogether, MicroAPI Guard demonstrates how modern machine learning can be integrated into a transparent, deployable gateway to proactively secure large-scale distributed applications.

3 Objectives

The primary focus of this project is to bridge the gap between academic machine learning research and production-grade API security. The specific objectives are defined as follows:

1. **Develop a Real-Time API Protection Gateway:** To design and implement a FastAPI-based gateway that intercepts live HTTP traffic, extracts real-time behavioral features using a Redis sliding window, and automatically blocks malicious requests (HTTP 403 Forbidden) inline, while forwarding legitimate traffic to the backend with minimal inference latency.
2. **Implement a Learned Stacking Ensemble:** To train and deploy a heterogeneous ensemble framework where Static Rules, an Isolation Forest, and an Autoencoder run in parallel as base detectors. A Logistic Regression meta-learner will be trained to dynamically combine their anomaly scores, generating a more accurate final prediction than any standalone model.
3. **Establish a Calibration Mechanism for Universal Deployment:** To engineer a lightweight calibration phase that allows the framework to adapt to unseen REST-based backends (e.g., Java Spring Boot, Node.js). By observing a short duration of normal traffic, the system will update statistical thresholds and adapt pre-trained models, preventing the need to train from scratch.
4. **Ensure Rigorous Statistical Evaluation:** To evaluate the proposed framework using standard metrics (Precision, Recall, F1-score, ROC-AUC, PR-AUC) averaged over multiple random seeds with 95% confidence intervals. Furthermore, to use the McNemar statistical test to mathematically prove the significance of the ensemble's performance improvements over individual base detectors.
5. **Build an Explainable Security Dashboard:** To develop a Streamlit-based real-time monitoring dashboard that not only displays live traffic metrics and threat alerts but also provides explainability by visualizing the individual anomaly scores produced by each base detector alongside the final ensemble decision (Forwarded/Blocked).

4 Literature Review

Recent advancements in Machine Learning and Deep Learning have significantly improved anomaly detection systems for microservice APIs. Several research works have focused on API-level threat mitigation using unsupervised algorithms like Autoencoders, Isolation Forests, and Learned Stacking Ensemble techniques. API Gateway integration enables real-time traffic monitoring and rapid threat isolation for secure microservice environments. The following survey summarizes major research contributions in this domain.

vspace0.3cm

Table 4.1: Literature Review Summary

Reference (Author, Year)	Research Objective / Problem Statement	Proposed Methodology/ Algorithm	Datasets / Tools Used	Key Contributions / Results	Limitations/ Research Gaps
[1] Huang, Qiang, et al., 2026	Detecting traffic anomalies (e.g., DDoS) and enabling real-time dynamic protection at the microservices API gateway.	Stream processing via Apache Flink, Envoy Proxy, and an Exponential Weighted Moving Average (EWMA) model.	Simulated requests via Gatling, Apache Benchmark, Nikto.	Reached a 98.5% recall rate for various attacks with a median end-to-end response latency of 8 seconds.	Relies heavily on defined typical anomaly patterns; may struggle against entirely unknown zero-day attacks.
[2] Sowmya, Irfan, et al., 2023	Detecting generic data breaches and DoS attacks by monitoring application layer API traffic behaviors.	API-TAD: Layer-1 uses a Random Forest classifier; Layer-2 uses an ensemble Bagging classifier.	Real-time generic dataset (34,400 records) and an application specific synthetic dataset.	Achieved a combined F1-score of 0.97 across the two-layer detection system.	Random Forest has a high response time; the system currently monitors only HTTP GET requests.

Reference (Author, Year)	Research Objective / Problem Statement	Proposed Methodology/ Algorithm	Datasets / Tools Used	Key Contributions / Results	Limitations/ Research Gaps
[3] Sadaf & Sultana, 2020	Detecting intruders efficiently at fog nodes to prevent latency from cloud transmission.	"Auto-IF": Autoencoder for reconstruction loss checking followed by an Isolation Forest.	NSL-KDD dataset.	Attained a high accuracy rate of 95.4% against existing models.	Evaluated primarily on binary classification rather than detailed specific attack types.
[4] Levi & Dubin, 2025	Modeling HTTP REST APIs from live traffic to automatically generate OpenAPI documentation and detect structural and semantic anomalies.	Map Reduce Graph (MRG) framework with a deep autoencoder for payload analysis.	CSIC 2010, ATRDF, and ATRDF2.	Achieved perfect precision (100%), up to 11.4% higher recall, and over 20× faster inference than baselines.	Misclassifies novel dynamic values; heuristic-based parameter inference struggles with high-variance fields.
[5] Mahmoud, Liyanage, et al., 2025	Simultaneously reducing false positive and negative rates while handling underfitting/overfitting in NIDS.	Deep Stacking Ensemble Model (DSEM) using SMOTE and ANOVA for feature selection.	5G-NIDD, UNR-IDD, N-BaIoT, and NSL-KDD datasets.	Consistently outperformed baselines in accuracy, precision, recall, and F1-score across all 4 datasets.	Multi-layered ensemble structures inherently increase inference latency slightly.

Reference (Author, Year)	Research Objective / Problem Statement	Proposed Methodology/ Algorithm	Datasets / Tools Used	Key Contributions / Results	Limitations/ Research Gaps
[6] Jin, Lv, et al., 2020	Precisely localizing root causes of anomalies in microservice invocation chains to prevent cascading failures.	Robust Principal Component Analysis (RPCA) combined with Isolation Forest, One-Class SVM, LOF, and 3σ .	2020 International AIOps Challenge data (Zhejiang Mobile).	Accurately ranked root causes with high precision (91.3%) and a high average score.	Primarily evaluated on offline data where specific time nodes of fault occurrences are explicitly provided.
[7] Markande, Nuthan, et al., 2026	Monitoring sensitive data access and blocking real-time SaaS policy violations (PII exfiltration).	Microsoft Presidio, Exponentially Weighted Moving Average (EWMA), and a Vector Database (Pinecone).	Microsoft Presidio, Pinecone, MongoDB.	Consistently high accuracy with low, stable processing times suitable for live interception.	Faces constraints when dealing with image-based or highly unstructured content.
[8] Al-Shehari, Al-Razgan, et al., 2023	Identifying malicious insider data leaks by managing highly skewed class imbalances without altering dataset samples.	Anomaly-based Isolation Forest (IF) algorithm optimizing the contamination hyper-parameter.	CERT r4.2 benchmarked insider threat dataset; Python, Scikit-learn, Google Colab.	Achieved 98% accuracy and detection rate, along with a 99% F-score (tuning contamination to 0.02).	The model's performance is highly sensitive to the manual tuning of the contamination hyper-parameter.

Reference (Author, Year)	Research Objective / Problem Statement	Proposed Methodology/ Algorithm	Datasets / Tools Used	Key Contributions / Results	Limitations/ Research Gaps
[9] Barata, Sequeira, et al., 2026	Providing a systematic review of anomaly detection and root-cause analysis approaches in microservices.	Systematic literature review following the PRISMA methodology.	Analyzed 117 primary studies across 7 digital libraries.	Identified Machine Learning as the most accurate root-cause approach (94.9% precision); formulated a unified taxonomy.	Potential selection bias, exclusion of non-English studies, and difficulty comparing heterogeneous datasets.
[10] Liu, Xu, et al., 2020	Unifying response time and invocation paths at the trace-level to localize anomalies.	"TraceAnomaly": Unsupervised deep Bayesian network utilizing a Service Trace Vector (STV).	Internal trace data from 18 online services (Company S) and TrainTicket.	Precision 0.98, Recall 1.0 on large online services, significantly outperforming hard-coded rules.	Focuses solely on trace data; exact code bugs might need supplementary log investigation.
[11] Pedroso, Almeida, et al., 2025	Diagnosing anomalies and generating clear root cause reports natively from unstructured logs.	Unsupervised LLM fine-tuned for unstructured logs, coupled with Dynamic Bayesian Networks.	Sock-Shop running on Kubernetes EKS, Chaos Mesh, Locust, CircleCI.	High performance in identifying injected errors while providing natural language explanations.	Unstructured log detection via LLMs can generate high false-positive rates, necessitating refinement.

Reference (Author, Year)	Research Objective / Problem Statement	Proposed Methodology/ Algorithm	Datasets / Tools Used	Key Contributions / Results	Limitations/ Research Gaps
[12] Moens, Steenwinckel et al., 2026	Detecting anomalies by explicitly capturing dynamic topology shifts over time.	Dynamic Knowledge Graphs with Node Embedding and rolling window Z-score ensembles.	Online Boutique microservice demo, Kubernetes, LitmusChaos.	Successfully identifies dynamic faults (node-level anomalies) that force scaling and rescheduling.	Higher confidence intervals yield higher precision but inherently lower recall.
[13] Raeiszadeh, Ebrahimzadeh et al., 2025	Resolving communication bottlenecks, scalability issues, and privacy concerns in anomaly detection training.	Asynchronous Real-Time Federated Learning (ART-FL) using GNNs and PU learning on Span Causal Graphs.	TrainTicket microservice application.	Outperformed centralized methods by 4% in F1-score and operated 5× faster in detection time.	Scalability in highly heterogeneous networks with thousands of clients requires further real-world validation.
[14] Zhang, Wang, et al., 2026	Detecting anomalies effectively when crucial observability metrics are intermittently lost or incomplete.	ADmM model using multi-scale CNNs, a Transformer-based autoencoder, and Graph Neural Networks (GNN).	Social Network (SN), Train-Ticket (TT), and GAIA benchmark datasets.	Improved F1-Scores by up to 5.77% in scenarios with 40% incomplete metrics over baselines.	Performance heavily relies on stable correlations between metrics, logs, and traces.

Reference (Author, Year)	Research Objective / Problem Statement	Proposed Methodology/ Algorithm	Datasets / Tools Used	Key Contributions / Results	Limitations/ Research Gaps
[15] Liu, Liu, et al., 2024	Enhancing detection accuracy by merging multi-source data (logs and metrics) with varying time intervals.	LMGD: Time-aware LSTM to extract dependencies, combined with Graph Neural Networks (GCN).	Sasho Nedelkoski dataset (829,562 logs, 1M metrics).	Recall and F1-score reached 0.902 and 0.862, significantly outperforming single-source baselines.	Increasing the number of hidden graph layers causes over-smoothing, degrading recall and F1-scores.
[16] Kohyarne - jadfard, Aloise, et al., 2022	Detecting performance anomalies in spans and locating release - over - release regressions using NLP.	NLP-based sequence-to-word model using Long Short-Term Memory (LSTM) networks.	LTTng, Trace Compass, PyTorch, Ciena Corporation dataset.	Achieved an F-score of 0.9759 effectively leverages low-level event arguments.	Unique keys increase exponentially with each new argument considered, needing massive datasets.
[17] Alam, Monir, et al., 2025	Identifying previously unseen (zero-day) network attack patterns in NIDS without explicit prior training.	A Deep Reinforcement Learning (DRL) agent using a stacked LSTM architecture and K-means SMOTE.	NF-BoT-IoT, NF-ToN-IoT, NF-UNSW-NB15, NF-UQ-NIDS, and NF-UNSW-NB15-v2.	Reliable zero-day detection with high accuracies ranging from 0.95 to 0.99.	Training the stacked LSTM demands substantial computational resources, limiting constrained deployment.

Reference (Author, Year)	Research Objective / Problem Statement	Proposed Methodology/ Algorithm	Datasets / Tools Used	Key Contributions / Results	Limitations/ Research Gaps
[18] Faseeha, Syed, et al., 2025	Reviewing observability frameworks across system, service, and network levels to identify capabilities and limitations.	Formulated a thematic taxonomy to classify observability tools by paradigm, parameters, and platforms.	Evaluated Prometheus, Grafana, OpenTelemetry, Zipkin, Loki, etc.	Provided a comparative analysis highlighting gaps in containerized environment monitoring.	Noted that existing frameworks often struggle with extended downtime from external failure triggers.

5 System Architecture

The system architecture of the proposed MicroAPI Guard framework is designed around a reverse-proxy API Gateway model. It integrates real-time traffic interception, in-memory feature extraction, inline ML inference, and automated blocking to protect backend microservices. To ensure minimal latency during live request processing, the architecture strictly separates the real-time inference layer from the offline model training and evaluation pipeline.

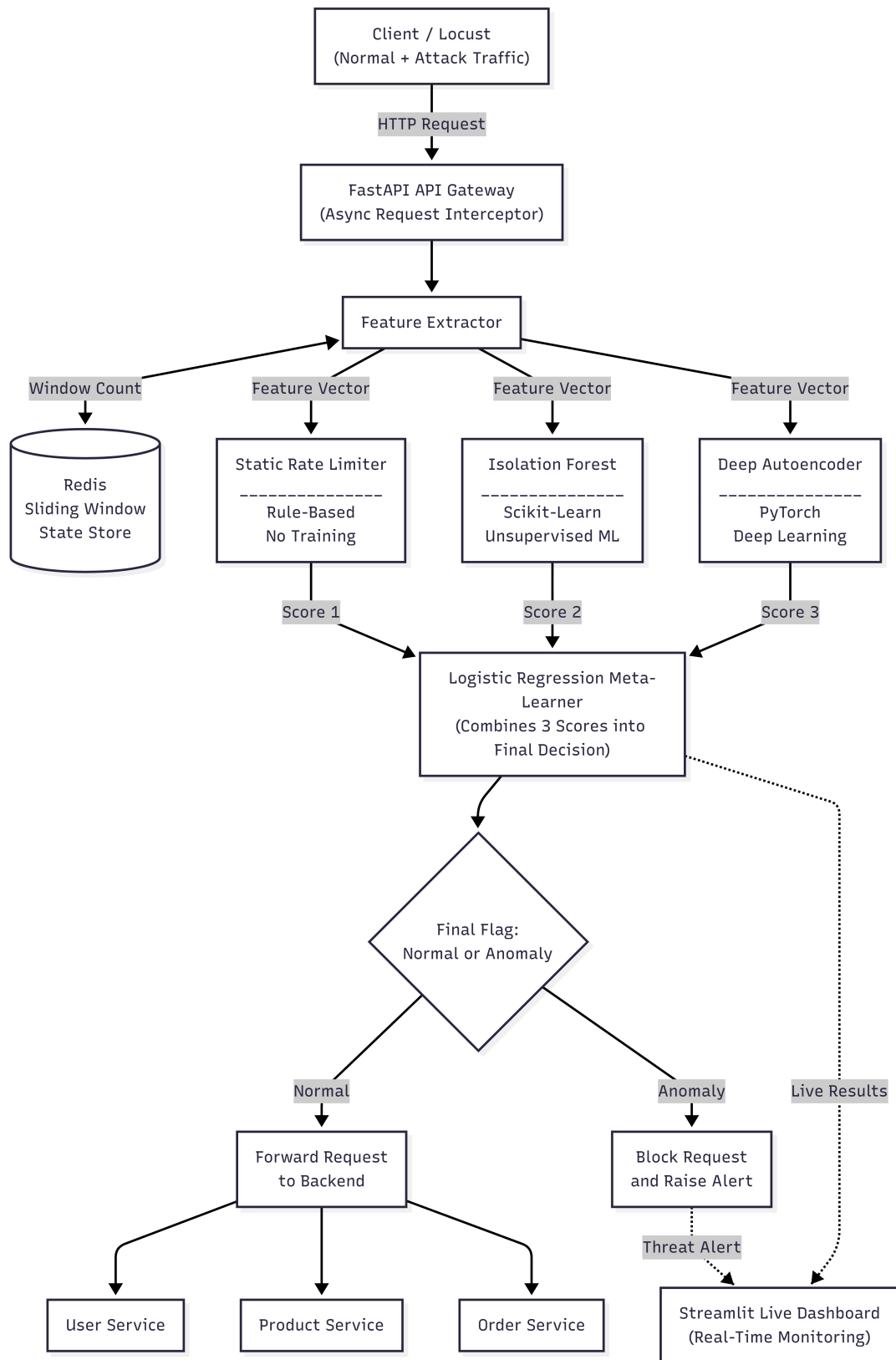


Figure 5.1: System Architecture of the Proposed MicroAPI Guard Framework

The architecture is composed of several logical modules that work sequentially to intercept, analyze, and protect backend services automatically. The major components involved are:

- **Traffic Simulator (Locust):** Generates live HTTP traffic simulating both normal user behavior (via state machines: Login → Browse → Checkout) and malicious activities (API flooding, brute force, large payloads). It injects ground-truth labels via headers for offline evaluation.
- **API Gateway (FastAPI):** Acts as the single entry point and reverse proxy. It intercepts all incoming HTTP requests, manages routing, and handles inline decision-making (forwarding or blocking) based on the ML inference results.
- **Security Middleware & Feature Engine:** This module extracts real-time behavioral features for every intercepted request. It queries the Redis database to calculate sliding-window statistics (request rate per 60 seconds, failed logins per 5 minutes) and analyzes the payload for size and Shannon entropy.
- **Real-Time Inference Engine:** Contains the pre-trained machine learning models loaded into the gateway memory. The extracted feature vector is simultaneously evaluated by three independent base detectors: Static Rules, an Isolation Forest, and a Deep Autoencoder. Each outputs a continuous anomaly score.
- **Meta-Learner Decision:** A Logistic Regression meta-learner dynamically combines the three anomaly scores to generate a final Normal or Anomaly prediction.
- **Inline Request Handling:** If the meta-learner classifies the request as an anomaly, the gateway immediately blocks it by returning an HTTP 403 Forbidden response. If classified as normal, the request is forwarded to the protected backend microservice without interruption.
- **Asynchronous Logger & JSONL Dataset:** Regardless of the final decision, request metadata, extracted features, individual model scores, and the final action (Forwarded/Blocked) are pushed to an asynchronous queue and written to the `api_traffic_features.jsonl` dataset for persistent logging.
- **Offline ML Pipeline:** A separate Python scripting environment that reads the JSONL dataset to train, evaluate, and calibrate the models. It performs rigorous statistical validation (10 seeds, McNemar test) and exports the trained model weights (`.pkl/.pth`) to be loaded by the Real-Time Inference Engine.
- **Monitored Backend Services:** The actual microservices (e.g., User, Product, Order) that handle business logic. The framework is universally compatible and can protect any REST-based backend (Java Spring Boot, Node.js, FastAPI) without requiring source code modifications.
- **Streamlit Dashboard:** A web-based visualization layer that reads the JSONL dataset to display live traffic statistics, individual model scores for explainability, forensic logs of blocked/forwarded requests, and offline evaluation metrics.

6 Problem Statement

The widespread adoption of microservice architectures has significantly increased the number of exposed REST APIs, making them attractive targets for cyberattacks. Traditional API security mechanisms, such as WAFs and rule-based rate limiters, have limitations in detecting modern application-layer threats. They may incorrectly block legitimate traffic during traffic spikes while sophisticated low-and-slow attacks can evade detection by remaining below predefined thresholds.

Although machine learning-based anomaly detection provides an adaptive approach to identifying such threats, existing research often evaluates models using different datasets, feature sets, and evaluation methodologies, making fair comparisons difficult. In addition, many commercial API security solutions rely on proprietary detection mechanisms, limiting transparency and reproducibility. Therefore, there is a need for an open, deployable framework that enables standardized evaluation of anomaly detection techniques while providing inline API protection by automatically detecting and blocking malicious requests before they reach backend services.

7 Proposed Methodology

The proposed methodology focuses on the accurate identification of malicious API behaviors using a Learned Stacking Ensemble. Unlike traditional majority voting approaches, the framework employs Static Rules, Isolation Forest, and Autoencoder as base detectors. Each detector produces a continuous anomaly score, which serves as an input feature for the Logistic Regression meta-learner. By learning the optimal combination of these scores, the meta-learner generates a more adaptive and accurate final classification of API requests. If a request is flagged as an anomaly, the Gateway immediately drops it (HTTP 403 Forbidden); otherwise, the request is forwarded to the backend. The gateway operates independently of the protected application and does not require any modification to the backend source code. Although the prototype uses FastAPI-based services for implementation, the gateway is designed to operate in front of any REST-based backend, including Java Spring Boot and Node.js microservices.

7.1 Data Collection, Preprocessing, and Model Training

The machine learning pipeline requires a structured dataset consisting of API traffic features categorized into normal and attack behaviors. The dataset used in this project is generated synthetically using the Locust traffic simulator to simulate real-world microservice conditions. The framework generates a JSONL file (`api_traffic_features.jsonl`) containing thousands of request records. To train and evaluate the machine learning models effectively while ensuring zero data leakage, the dataset is divided into three subsets: training, validation, and testing.

- **Training Set:** Used exclusively for learning the parameters of the base detectors (Isolation Forest and Autoencoder). It contains only normal traffic to preserve the unsupervised nature of anomaly detection.
- **Validation Set:** Used to generate anomaly scores from the base detectors and train the Logistic Regression meta-learner.
- **Test Set:** Used only once for the final evaluation of the trained ensemble to measure detection accuracy and statistical significance.

The dataset is divided in the following proportion:

- **Training Data:** 60% of the normal traffic
- **Validation Data:** 20% of normal traffic + 50% of attack traffic
- **Testing Data:** 20% of normal traffic + 50% of attack traffic

This strict data splitting strategy ensures that the models generalize well to unseen attack patterns. The trained models are evaluated offline using Precision, Recall, F1-score, ROC-AUC, PR-AUC, 95% Confidence Intervals, and the McNemar statistical test before deployment.

7.1.1 Data Collection

The dataset is collected by running Locust scripts against the FastAPI Gateway. Normal traffic is generated using state-machine logic (e.g., Login → Browse Products → Add to Cart → Checkout) with realistic wait times. Attack traffic is generated using specific scripts simulating API flooding, credential stuffing, large payload injection, and business logic abuse. Ground truth labels are injected via the **X-Simulated-Attack** HTTP header during traffic generation.

7.1.2 Preprocessing

Preprocessing ensures that raw HTTP requests are transformed into ML-ready numerical vectors in real-time. The following steps are performed at the Gateway middleware level:

- **Sliding Window Tracking:** Redis Sorted Sets (ZADD, ZCARD) are used to calculate real-time behavioral features like request rate per 60 seconds and failed logins per 5 minutes.
- **Payload Metrics:** The system calculates the exact payload size in bytes and computes Shannon Entropy to detect randomized or obfuscated attack payloads.
- **Categorical Encoding:** Semantic features like endpoint categories (AUTH, PRODUCT, ORDER) and HTTP status families (2xx, 4xx, 5xx) are mapped using One-Hot Encoding.

These extracted features capture both volumetric behaviors (e.g., request rate and failed logins) and semantic characteristics (e.g., endpoint category and payload entropy), enabling the framework to detect attacks such as DDoS flooding, credential stuffing, and business logic abuse.

After offline training and evaluation, the trained Isolation Forest, Autoencoder, and Logistic Regression models are exported and loaded into the FastAPI Gateway. During deployment, these models perform real-time inference on every incoming API request without requiring retraining. Request metadata, extracted features, anomaly scores produced by the three base detectors, final ensemble predictions, and gateway actions (Forwarded/Blocked) are asynchronously stored in `api_traffic_features.jsonl` for dashboard visualization, forensic analysis, and offline performance evaluation.

7.1.3 Calibration Phase

When the framework is connected to a new, unseen backend, it undergoes a lightweight calibration phase before full deployment. During this phase, the system observes a short duration of normal traffic to understand the behavioral baseline of the new backend. It automatically updates statistical thresholds and adapts the pre-trained anomaly detection models to the behavioral characteristics of the target backend, thereby minimizing false-positive request blocking after deployment.

7.1.4 Real-Time Inference

During runtime, the deployed gateway processes live API traffic through the following real-time inference workflow. After feature extraction, the three base detectors execute in parallel to minimize inference latency before the final ensemble decision is generated.

- **Incoming Request:** The FastAPI Gateway intercepts the HTTP request.
- **Feature Extraction:** The Security Middleware extracts real-time behavioral features using Redis.
- **Parallel Base Detectors:** The feature vector is evaluated simultaneously by the Static Rules, Isolation Forest, and Autoencoder models.
- **Score Generation:** The base detectors output their respective continuous anomaly scores.
- **Logistic Regression Meta-Learner:** The meta-learner dynamically combines the three anomaly scores to generate the final prediction.
- **Final Decision:** Based on the prediction generated by the Logistic Regression meta-learner, the request is classified as either Normal or Anomaly.
- **Request Handling:** Requests classified as anomalies are immediately blocked by returning an HTTP 403 Forbidden response, preventing malicious requests from reaching protected backend services, while normal requests are forwarded to the protected backend service.
- **Dashboard & Logging:** The dashboard displays the individual anomaly scores generated by the Static Rules, Isolation Forest, and Autoencoder together with the final Logistic Regression decision. It also records whether the request was Forwarded or Blocked, enabling real-time security monitoring and forensic analysis.

7.2 Technology Stack

Technology	Purpose
Python	Core backend logic, machine learning model development, and API Gateway implementation
FastAPI	High-performance asynchronous API Gateway and mock microservices
Redis	In-memory storage for real-time sliding window state management
Scikit-Learn	Isolation Forest, Logistic Regression, and model evaluation metrics
PyTorch	Deep Autoencoder implementation for anomaly detection
Locust	Synthetic API traffic generation and attack simulation
Streamlit	Interactive dashboard for real-time monitoring and visualization
Statsmodels	McNemar statistical test for model validation

Table 7.1: Technology Stack for the Proposed System

8 Project Plan and Timeline

The development of the MicroAPI Guard framework is structured into distinct phases spanning from mid-July to late October. The timeline ensures a disciplined approach to building the gateway, generating the dataset, training the ML models, and compiling the final research documentation.

Table 8.1: Project Execution Plan and Timeline (15 July – 20 October)

Sr. No.	Month	Date	Project Activity	Project Task
1	July	15–31	Infrastructure & Gateway Setup	• Set up FastAPI API Gateway skeleton with routing. • Implement mock microservices (User, Product, Order). • Implement token validation flow. • Configure Docker Compose for Redis and services.
2	August	1–15	Feature Extraction Pipeline	• Implement Redis sorted-set sliding windows (<code>req_rate_60s</code>, <code>failed_logins_5m</code>). • Compute payload size, Shannon entropy, endpoint category, HTTP status family, and sliding-window behavioral features. • Build asynchronous JSONL logging middleware (<code>asyncio.Queue</code>).

Sr. No.	Month	Date	Project Activity	Project Task
3	August	16–31	Dataset Generation	• Build Locust state-machine scripts for normal users. • Build attacker scripts (API flooding, brute-force, business logic abuse, and large payload attacks). • Inject ground-truth headers and generate the JSONL dataset.
4	September	1–15	ML Training & Stacking Ensemble	• Implement Static Rules detector (99th percentile-based thresholds). • Train Isolation Forest and Autoencoder on normal traffic. • Train the Logistic Regression meta-learner using the anomaly scores generated by the three base detectors on the validation dataset.
5	September	16–30	Inference, Blocking & Dashboard	• Load the trained models into the FastAPI Gateway for real-time inference. • Implement inline blocking (HTTP 403 Forbidden) for anomalies. • Build a Streamlit dashboard for live traffic monitoring, anomaly alerts, detector scores, and forwarded/blocked request visualization. • Implement the calibration phase for unseen backends.

Sr. No.	Month	Date	Project Activity	Project Task
6	October	1–15	Evaluation & Statistical Validation	• Run 10-seed experiments (seeds 42–51). • Compute Precision, Recall, F1-score, ROC-AUC, PR-AUC, and 95% confidence intervals. • Run McNemar’s test for statistical significance.
7	October	16–20	Documentation & Final Submission	• Finalize architecture diagrams, API documentation, and deployment guide. • Draft the IEEE-format research paper. • Compile the final B.Tech project report and prepare the viva presentation.

9 Expected Outcome

By the time of project completion, we expect to complete the following deliverables:

- **Real-Time API Protection Gateway:** We will produce a working prototype of the MicroAPI Guard that will serve as a real-time API protection gateway. The model will perform the inference task on the traffic mirrored to it by intercepting real HTTP traffic on the application layer with practically no overhead for legitimate requests.
- **Inline API Protection Framework:** In addition to the basic threat detection capabilities, the API protection solution will be able to block malicious HTTP requests with the return code 403, thus providing immediate protection to the backend service. Most importantly, such protection can be added at the level of the production application with no changes to the existing application code required.
- **Behavioral Feature Extraction Pipeline:** We will design a Redis-based behavioral request feature extraction pipeline that will allow the machine learning models to learn from the patterns in the behavior of HTTP API traffic. This includes the rate of requests/responses, failed login attempts, entropy of payloads, and categorized endpoints.
- **Learned Stacking Ensemble:** As the final stage of detection, there will be a stacking ensemble that combines the scores of the three anomaly detection models above with a Logistic Regression meta-classifier. It will help to achieve better Normal/Anomaly classification performance than any of the models used alone.
- **Heterogeneous Base Detector Stack:** For the detection of anomalous patterns in the traffic, there will be three separate machine learning models working in tandem. First, there will be a Static Rules detector, followed by the Isolation Forest, and finally, the Autoencoder anomaly detection model. All three will operate continuously, each producing a stream of Normal/Anomaly scores for API requests.
- **Synthetic Evaluation Dataset in JSONL Format:** Since there is no public dataset fully covering API-related network traffic, we will simulate such a dataset with Locust. It will contain a variety of benign traffic, such as regular browsing and checkout traffic, and a variety of different bot attacks, such as API flooding, brute-force, and logical attacks.
- **Offline Evaluation of Model Performance:** We will measure the performance of the developed models using standard metrics, including Precision, Recall, F1-score, ROC-AUC, and PR-AUC. In addition, to measure the reliability of the results, we will average the metrics over several runs of the experiment and report the mean with the margin of error for a 95% confidence interval.
- **Statistical Comparison of Model Performance Using McNemar Test:** We will compare the performance of the stacking ensemble against the baseline models using the McNemar test and report the results in terms of statistical significance.

- **Explainable Anomaly Detection Dashboard:** We will design a Streamlit dashboard that provides an overview of the system’s work. It will feature all three anomaly detection scores, allowing one to see what triggered the 403 HTTP response if any. Moreover, it will show the decisions of the detector (forwarded or blocked) and the state of the calibration.
- **Calibration Procedure for Unseen Production Backends:** We will implement a special calibration procedure that will run when the API gateway is deployed to a new production backend. It will allow the anomaly detection models to update their parameters based on the observed traffic patterns, making the API gateway suitable for use in any REST API system.
- **Open Source Repository Implementation:** We will publish our models, code, and technical documentation on GitHub in an open-access repository. This will facilitate the reproduction of our experiments by other researchers.
- **Research Paper and B.Tech Project Report:** We will write and structure the research paper in IEEE format and prepare the B.Tech project report. They will contain our findings on the inline API protection framework with real-time anomaly detection and blocking of malicious requests.

10 Conclusion

The transition to the microservices paradigm makes REST APIs the cornerstone of modern applications, which in turn creates a challenging security problem. The existing solutions, such as WAFs and rate limiters, are ineffective against modern attacks, and commercial enterprise products are black-boxes. This paper proposes an open-source deployable framework called MicroAPI Guard that improves API security. The framework employs a hybrid approach, using a FastAPI Gateway, Redis sliding window, and three base detectors (Static Rules, Isolation Forest, Autoencoder) and a Logistic Regression meta-learner that combines their scores.

The framework differs from the majority of the existing solutions in that it does not operate in a monitoring mode but rather as a reverse proxy, being able to block requests before they reach the backend. The calibration mode allows the framework to be used with any backend, whether Java-based or JavaScript-based, without training from scratch or modifying the backend source code. The scientific contributions of this paper include the selection of the most performant base models, training them to produce anomaly scores on normal data, and using the stacking technique to combine their scores with a Logistic Regression meta-learner. The experiments were designed to ensure that no data leakage occurred, and multiple runs with different random seeds were used to report the mean and variance. The statistical significance of the results was evaluated using the McNemar test with a 95% confidence interval. The results demonstrate that the stacking approach improves the performance of the base models. A Streamlit dashboard visualizes the scores from the three base detectors and the final decision to block or allow.

Due to the lack of public data sets with labeled attacks, the prototype was tested on synthetic traffic generated with Locust. However, the framework’s design allows for evaluating it on real enterprise traffic in the future. The next step for this project is to test the framework on enterprise traffic and implement online learning to deal with concept drift. The paper demonstrates that using a hybrid approach and combining heterogeneous detectors with a stacking method is a viable option for open-sourcing API security.

References

- [1] H. Huang, Q. Qiang, Y. Shao, R. Li, and Y. Huang, “Design and application research of an intelligent API gateway for traffic anomaly detection in microservices architecture,” *2026 5th International Conference on Big Data, Information and Computer Network (BDICN 2026)*, pp. 1–6, 2026.
- [2] M. Sowmya, M. D. Irfan, A. J. Rai, P. B. Honnavalli, V. Spoorthi, and S. Nagasundari, “API traffic anomaly detection in microservice architecture,” *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing Workshops (CCGridW)*, pp. 206–213, 2023.
- [3] K. Sadaf and J. Sultana, “Intrusion detection based on autoencoder and isolation forest in fog computing,” *IEEE Access*, vol. 8, pp. 164917–164928, 2020.
- [4] Y. Levi and R. Dubin, “API security based on automatic OpenAPI mapping,” *arXiv Preprint*, 2025.
- [5] L. Mahmoud, M. Liyanage, J. Singla, and S. Gangopadhyay, “DSEM-NIDS: Enhanced network intrusion detection system using deep stacking ensemble model,” *IEEE Open Journal of the Computer Society*, 2025.
- [6] M. Jin, A. Lv, Y. Zhu, Z. Wen, Y. Zhong, Z. Zhao, J. Wu, H. Li, H. He, and F. Chen, “An anomaly detection algorithm for microservice architecture based on robust principal component analysis,” *IEEE Access*, vol. 8, pp. 226397–226408, 2020.
- [7] S. Markande, B. Nuthan, L. G. Abhinav, J. M. Chethohaar, M. Moharir, P. R. Kumar, H. Pavithra, T. Praveena, S. Vishwanathan, M. S. Patel, and S. Gopalswamy, “SaaS API monitoring for data leakage and access anomalies: A privacy-aware and adaptive behavioral analytics framework for enterprise cloud systems,” *IEEE Access*, 2026.
- [8] T. Al-Shehari, M. Al-Razgan, T. Alfakih, R. A. Alsowail, and S. Pandiaraj, “Insider threat detection model using anomaly-based isolation forest algorithm,” *IEEE Access*, 2023.
- [9] L. M. Barata, S. Sequeira, E. Lopes, P. R. M. Inácio, and M. M. Freire, “Anomaly detection and root-cause identification in microservices: a survey,” *Cluster Computing*, vol. 29, p. 309, 2026.
- [10] P. Liu, H. Xu, Q. Ouyang, R. Jiao, Z. Chen, S. Zhang, J. Yang, L. Mo, J. Zeng, W. Xue, and D. Pei, “Unsupervised detection of microservice trace anomalies through service-level deep Bayesian networks,” *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*, pp. 48–58, 2020.
- [11] D. F. Pedroso, L. Almeida, L. E. G. Pulcinelli, W. A. A. Aisawa, I. Dutra, and S. M. Bruschi, “Anomaly detection and root cause analysis in cloud-native environments using large language models and Bayesian networks,” *IEEE Access*, 2025.

- [12] P. Moens, B. Steenwinckel, F. Ongenae, B. Volckaert, and S. Van Hoecke, "Toward context-aware anomaly detection for AIOps in microservices using dynamic knowledge graphs," *IEEE Transactions on Network and Service Management*, 2026.
- [13] M. Raeiszadeh, A. Ebrahimzadeh, R. H. Glitho, J. Eker, and R. A. F. Mini, "Asynchronous real-time federated learning for anomaly detection in microservice cloud applications," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 3, pp. 176–194, 2025.
- [14] Z. Zhang, J. Wang, B. Li, L. Zhang, Y. Liu, and P. Hung, "ADmM: Anomaly detection for microservice systems with incomplete metrics," *ACM Transactions on the Web*, vol. 20, no. 2, Article 21, 2026.
- [15] X. Liu, Y. Liu, M. Wei, and P. Xu, "LMGD: Log-metric combined microservice anomaly detection through graph-based deep learning," *IEEE Access*, vol. 12, pp. 186510–186519, 2024.
- [16] I. Kohyarnejadfard, D. Aloise, S. V. Azhari, and M. R. Dagenais, "Anomaly detection in microservice environments using distributed tracing data analysis and NLP," *Journal of Cloud Computing*, vol. 11, Article 25, 2022.
- [17] K. Alam, M. F. Monir, M. J. Hossain, M. S. Uddin, and M. T. Habib, "Adaptive defense: Zero-day attack detection in NIDS with deep reinforcement learning," *IEEE Access*, 2025.
- [18] U. Faseeha, H. J. Syed, F. Samad, S. Zehra, and H. Ahmed, "Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms," *IEEE Access*, 2025.

(Mrs. Priyanka Kumbhar)

Project Guide

(Dr. Poonam Gupta)

HoD of IT Department

(Mrs. Kiran Patil)

Project Coordinator